ForgeTrain: Forging Production-Grade Training Frameworks via Harness-Driven AI Development

Zhui Zhu **Qingfeng He** **Shangzhan Li** **Yaojian Chen**
Haojun Sun Zhen Li Yuxuan Li Zhiyuan Liu
ModelBest Inc.
{zhuzhui, heqingfeng, lishangzhan, chenyaojian,
sunhaojun, lizhen, liyuxuan, liuzhiyuan}@modelbest.cn

Abstract

Training large models still rarely escapes general-purpose frameworks such as Megatron-LM, but generality is never a free asset—it exacts two costs on every scenario: a single framework serving all scenarios at once must adopt design compromises no individual scenario would choose, putting scenario-specific peak performance out of reach; and the layers of abstraction and configuration dispatch accumulated to accommodate them all erode MFU at runtime, step by step. AI code generation, meanwhile, has scaled from functions to entire runnable systems, driving the cost of forging a framework from scratch toward zero—yet no AI-forged training framework has matched its human reference. We present **ForgeTrain**, a methodology that forges a dedicated, disposable framework for each scenario against a trusted framework held as a *golden reference*. Correctness and performance, which no single target satisfies at once, are separated in time along a monotonically relaxing equivalence lattice—*Anchor*, *Bit-for-Bit*, *Surpass*—while a knowledge prior and recursively decomposed Milestones supply the optimization knowledge and the reachable curriculum an autonomous agent loop otherwise lacks. The forged code is regenerated per scenario; the forging apparatus, not the code, is the lasting asset—an inversion we call *Forge Engineering*. From an empty repository, ForgeTrain forges per-scenario engines for MiniCPM4 0.5B and 8B that reproduce the reference bit for bit and then surpass it by 5–10% in throughput at matched training quality, the 0.5B engine in roughly ten hours. To our knowledge this is the first production-grade training framework forged end-to-end by AI to match or surpass its human reference on the reference's own benchmarks, and concrete evidence of the L3→L4 transition on the infrastructure axis of progress toward AGI.

1 Introduction

Training a frontier model has become one of the most capital-intensive activities in all of computing: the final training run of a frontier model already costs tens of millions of dollars and is headed toward $1B per run [Cottier et al., 2024], with industry-wide capital committed to AI infrastructure in 2026 on the order of $700B [TechInsights, 2026]. The software that converts this capital into a trained model—the *training framework*—is no peripheral component: it fixes the throughput, the memory footprint, and the model-FLOPs utilization (MFU), so every point of efficiency left on the table translates directly into millions wasted on a single run. Infrastructure engineering is, moreover, a direct constituent of frontier capability: DeepSeek-V3 trained on a fraction of its peers' compute budget precisely through training-stack engineering—FP8 training, compute–communication overlap, cross-node communication kernels [DeepSeek-AI, 2024]—and software-side efficiency gains match hardware's—the compute needed to reach a fixed performance level halves roughly every eight

Preprint. Under review.

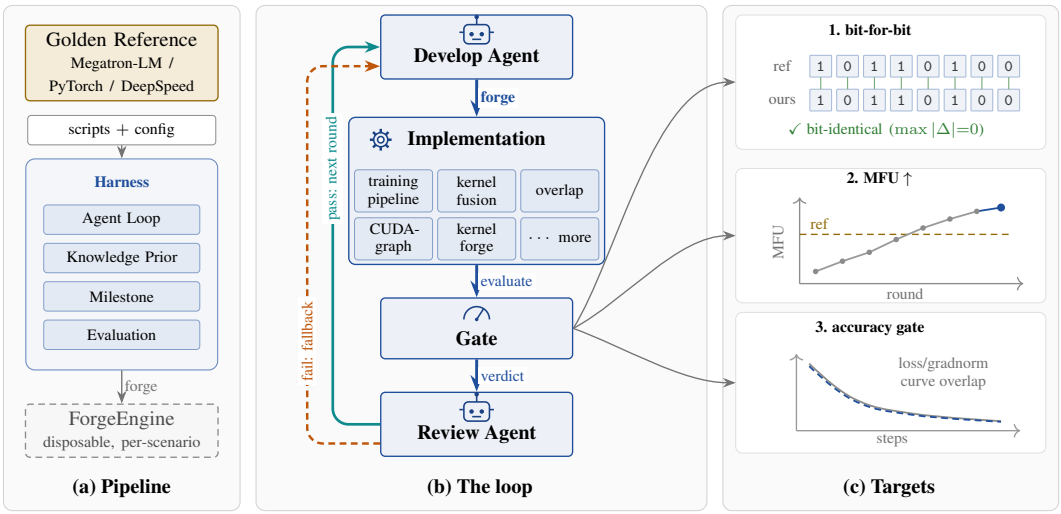

Figure 1: **ForgeTrain at a glance.** **(a)** *Pipeline*: a *golden reference* (Megatron-LM) and a thin per-scenario configuration enter the *Harness*—the durable, reusable apparatus built from three parts (*Agent*, *Evaluation*, *Prompt*)—which forges a disposable, per-scenario *ForgeEngine*. **(b)** *The loop* runs top-down. The develop agent *forges* an Implementation—assembling the training pipeline and applying kernel fusion, compute/communication overlap, kernel forging, and more; the *Gate* (the codified Evaluation) evaluates the forged candidate; a review agent issues a verdict; control then returns to the develop agent—advancing to the *next round* on pass, or *falling back* on fail. **(c)** *Targets*: the Gate checks three goals—**(1)** bit-for-bit identity with the reference, then **(2)** higher MFU at **(3)** a training-loss curve that stays on top of the baseline—so the loop first reproduces the reference exactly and then surpasses its throughput while preserving training quality.

months [Ho et al., 2024]—so how wide that channel runs depends directly on the quality of the framework.

Yet this leverage goes largely uncollected today. Training a large model today hardly avoids a general-purpose training engine. Take Megatron-LM [Shoeybi et al., 2019]: a codebase of a hundred thousand lines that has sedimented abstraction layers, configuration switches, and special-case branches for every model, scale, and device it has ever served, while for any one concrete model-and-hardware pair only a small slice of it carries the performance. This generality was never a free asset but exacts two distinct costs on every scenario. The first is *suboptimality*: because a single framework must serve all scenarios simultaneously, its design necessarily reflects compromises that no individual scenario would adopt in isolation—the tensor sharding, memory layout, operator fusion, and communication schedule that would be optimal for the case at hand are constrained to remain viable across the others, so scenario-specific peak performance is unattainable. The second is *overhead*: because the framework must accommodate an ever-growing set of scenarios, it accumulates successive layers of abstraction, indirection, and configuration dispatch, and these layers impose runtime cost that erodes MFU at every step. The heavier the inherited baggage, the more pronounced both effects become. The tax falls on human engineers and agents alike: handing an agent a debt-laden monolith is no easier than letting it start over.

The general framework was defaulted to on one economic fact: building a framework from scratch was prohibitively expensive, so reuse-and-extend was the only rational path. That premise is now dissolving: coding agents have begun synthesizing entire runnable systems on their own—a fully AI-generated deep-learning runtime [Xu et al., 2026], a hundred-thousand-line compiler [Anthropic, 2026]—and though neither reaches production grade, they already drive the cost of forging a framework from scratch toward zero, so reuse-and-extend no longer holds an automatic advantage. Once that is true, the better move is to forge a dedicated framework from scratch for each scenario: fitted natively to its model and hardware, free of inherited baggage on the way to the peak, exempt from both taxes at once. The significance also runs deeper: the training framework is the last major piece of the AI-building toolchain still human-authored—when the field measures automated-R&D capability on the road to AGI, the tasks it sets are exactly such infrastructure-engineering tasks:

optimizing training scripts, writing GPU kernels, fixing distributed training [Wijk et al., 2024]—so forging it is a necessary link on the way to AGI (§2.1). Yet forging such a framework with an agent is, in itself, extraordinarily hard, and the difficulty is intrinsic to the artifact and the task rather than to any externally imposed bar—matching or surpassing the reference is merely the target the forged framework must hit, not the source of its difficulty.

Forging such a framework comes down to three questions—how to keep it *correct*, how to push it to *peak performance*, and whether such a framework *can be optimized into existence at all*. We present **ForgeTrain**, a methodology whose answer to all three flows from a single organizing insight: *do not define correctness from scratch—appropriate an implementation already known to be correct, and hold it as ground truth.* We take any trusted, correct framework—such as Megatron-LM [Shoeybi et al., 2019], PyTorch [Paszke et al., 2019], or DeepSpeed [Rajbhandari et al., 2020]—as a *golden reference*: an artifact we hold as ground truth—it *defines* what is correct, and serves at once as *oracle* and *yardstick*. We forge the new framework against it, first matching it *bit for bit* and only then diverging to surpass it. The forging itself runs as an *agent loop*: an autonomous coding agent generates and explores candidate implementations and optimizations, each adjudicated against the golden reference. The golden reference and the agent loop run through all three answers that follow. Against that anchor, each question has its own difficulty and its own answer:

- **How to maintain correctness?** Correctness here is the strongest possible notion—*bit-level agreement* with the reference—yet being worth the effort demands *diverging* from it to surpass it; no fixed target is at once "equal to" and "better than" the reference. ForgeTrain separates the two demands *in time* along a three-stage spine—*Anchor* captures the reference's behavior, *Bit-for-Bit* reproduces it exactly with optimization forbidden, and *Surpass* relaxes the equivalence to training-quality parity—and breaks each climb into short, reference-verified steps with *Milestones*, so correctness is held continuously at every waypoint rather than accepted once at the finish line.

- **How to reach peak performance?** The peak is no single leap: the bar is set at an MFU target far above the current implementation, and no one optimization clears it. ForgeTrain adjudicates every candidate of the multi-round sprint with a *Gate*—a real training run comparing training quality and throughput at once—where failure returns an actionable discrepancy and passage locks the gain into the baseline, ratcheting forward only, so measured performance climbs toward the target round by round with every percentage point backed by an adjudicated run.

- **Can such a framework be optimized into existence?** The rewrites that approach the peak are multi-step, with worse intermediate states; a bare loop relaxes into the nearest local optimum before the gain appears—a shortfall of nerve rather than knowledge, which more code cannot fix. ForgeTrain supplies a *knowledge prior*—a corpus of optimization experience the agent consults—that raises the search's effective learning rate, emboldening the agent to commit to the multi-step optimizations it would otherwise abandon.

The forged framework's code is disposable, regenerated per scenario and never maintained, while the apparatus that produces it—the staged discipline together with the knowledge prior and Milestones—persists and is reused across every framework we forge. We call the resulting inversion—code as disposable output, the forging apparatus as the lasting asset—*Forge Engineering* (Figure 1): the natural equilibrium once the marginal cost of writing code approaches zero, just as falling design cost once pushed hardware from general-purpose CPUs toward domain-specific architectures [Hennessy and Patterson, 2019].

Faithful to that discipline, ForgeTrain is not one framework generalized across models but a methodology that forges a dedicated implementation—which we call a **ForgeEngine**—for each target. From an empty repository it forges a separate ForgeEngine for MiniCPM4 0.5B and for MiniCPM4 8B—the 0.5B engine in roughly *ten hours*, set against the multi-year effort of the dedicated team behind Megatron-LM—and we then train MiniCPM4 0.5B from scratch with its forged engine as end-to-end validation.

Concretely, we make three contributions:

- We identify the two structural costs a general-purpose training framework exacts on every model-and-hardware scenario—design suboptimality born of serving all scenarios at once, and runtime overhead accumulated to accommodate them—and argue that once coding agents drive the cost of forging a framework from scratch toward zero, forging a dedicated framework per scenario

becomes the better path to peak performance; code thereby becomes a disposable artifact and the forging apparatus the lasting asset, an inversion we call *Forge Engineering*.

- We propose the *golden reference* method to walk that path: appropriate a trusted, correct implementation as at once oracle and yardstick, and resolve, along a monotonically relaxing equivalence lattice—from bit-for-bit agreement down to training-quality parity—the two demands no single target satisfies at once: equaling the reference and beating it. ForgeTrain instantiates the method with the three-stage spine and Milestones, the Gate's round-by-round measured adjudication, and the knowledge prior.
- We deliver ForgeEngine, per-scenario frameworks forged end-to-end by ForgeTrain that match Megatron-LM on training quality and surpass it by 5–10% in throughput, and we train MiniCPM4 0.5B from scratch with its forged engine to downstream-evaluation parity with a Megatron-trained baseline—to our knowledge the first production-grade training framework forged end-to-end by AI to match or surpass its human reference on the reference's own benchmarks.

The remainder of the paper proceeds as follows. §2 situates ForgeTrain within the AGI capability levels and the Harness Engineering consensus. §3 develops the ForgeTrain methodology: the golden-reference refinement, the three-stage spine with Milestones that secures correctness, the Gate that drives the throughput climb, and the knowledge prior. §5 reports throughput, training quality, and end-to-end validation against Megatron-LM. §6 reviews adjacent work in large-scale training frameworks and AI-generated systems and kernels. §7 closes with implications for the recursive self-improvement frontier.

2 Background

The introduction argued that forging a training framework from scratch, per scenario, has become economically viable—and that the task itself is extraordinarily hard. This section lays down three layers of coordinates for that task: first the AGI capability levels, placing the task on the ladder and showing why it is a necessary crossing point to the next level (§2.1); then the autonomous agents holding the frontier today, showing what they have automated and what they leave to humans (§2.2); and finally Harness Engineering, the consensus methodology for driving AI to produce reliable code, together with an unstated assumption of it that the present work removes (§2.3).

2.1 AGI Capability Levels

Answering how much forging a training framework matters—and where it is stuck today—requires a coordinate system that *stratifies* progress in AI capability. OpenAI introduced a ready-made five-level framework for tracking progress toward AGI [OpenAI, 2024]: it ascends by the complexity of the tasks AI can carry, from conversational chatbots at L1 to systems that do the work of an entire organization at L5 (Table 1). We borrow it not to judge AGI timelines, but to place the task of forging a training framework on this ladder and make "which level does it belong to, and where is it stuck today" an answerable question.

Table 1: OpenAI's five-level framework for tracking progress toward AGI [OpenAI, 2024].

Level	Name	Defining characteristic
L1	Chatbots	Conversational language
L2	Reasoners	Human-level problem solving
L3	Agents	Systems that can take actions
L4	Innovators	AI that can aid in invention
L5	Organizations	Doing the work of an entire organization

Read in the setting of AI research and development, the decisive boundary is the L3-to-L4 transition: the *object of modification lifts upward*—from changing the model to changing the training pipeline, the evaluation, and the improver itself—the point widely identified as the recursive self-improvement takeoff line [Amodei, 2024]. Decomposed along the conventional triad of algorithm, infrastructure, and data, crossing it requires all three axes to lift to the paradigm level, and infrastructure is precisely where a training framework lives. Agents today close end-to-end loops within a fixed paradigm (L3), but the improver's own infrastructure—the training framework—stays human-written, holding the

frontier below L4 on the infrastructure axis. Forging that framework is what crossing into L4 there requires. And to cross, the first question is how far today's agents have actually come.

2.2 Autonomous Agents for AI R&D

The frontier is held today by *autonomous agents*: language models wrapped in a control loop that lets them pursue multi-step goals with diminishing supervision. The engine of every such agent is the same observe–act–reflect loop. Building on chain-of-thought reasoning [Wei et al., 2022] and learned tool use [Schick et al., 2023], ReAct interleaves reasoning traces with actions [Yao et al., 2023], Reflexion adds a verbal self-correction memory that carries lessons across trials [Shinn et al., 2023], and Voyager shows an ever-growing skill library compounding competence in an open-ended environment [Wang et al., 2023]. This loop is what turns a static model into the multi-day coding agents that now navigate codebases, run commands, and iterate against test feedback.

Pointed at AI research itself, the same paradigm yields its most consequential application: *automated AI research*. Where automated machine learning and neural architecture search once automated narrow slices of the algorithm axis [Zoph and Le, 2017]—evolutionary search evolved complete algorithms from primitive mathematical operations [Real et al., 2020], program search discovered optimizers deployed in production training [Chen et al., 2023], and agentic systems now conduct autonomous architecture discovery [Liu et al., 2025]—agentic systems today drive a far wider loop—The AI Scientist runs ideation, experiment implementation, execution, and manuscript writing in a single open-ended pipeline that emulates the research lifecycle [Lu et al., 2024]. This is the closest any existing system has come to the L4 boundary that §2.1 drew.

Yet however far the loop is pushed, it runs *on top of* a toolchain it does not author: an autonomous agent automates the *use* of training frameworks, evaluators, and kernels, not their construction. Even automated AI research improves the model and the experiment while the training framework beneath it stays human-written. This is exactly the boundary §2.1 placed on the infrastructure axis: the loop is automated, but the improver's own infrastructure is not. To *author* that infrastructure rather than merely operate it, an agent first needs an apparatus that can adjudicate its output—which brings us to the consensus methodology for driving AI to produce reliable code.

2.3 Harness Engineering

By 2026, Harness Engineering had become the consensus answer to the question of *what AI must be driven by* to produce reliable code at scale. OpenAI introduced the label *harness engineering* for long-horizon systems whose reliability rests on a durable specification and its automated judgment rather than on prompt wording [OpenAI, 2026]. Anthropic frames the complementary notion of an *agent harness*, the scaffold that turns a model into an agent, and argues that evaluating an agent means evaluating the harness and the model together rather than the model in isolation [Anthropic, 2025]. Across both formulations a Harness is an executable specification paired with an evaluation suite: it encodes what the desired behavior is and how to judge whether a candidate implementation achieves it, letting an agent iterate against automated judgment rather than human review. Public benchmarks make the structure concrete: SWE-bench, for instance, grades a model's patch to a real repository by executing that repository's own test suite [Jimenez et al., 2024]—precisely the specification-plus-evaluation pairing a Harness encodes. This is the feedback loop that converts a model's general competence into directed progress on a specific target. And it has already been proven at scale: OpenAI's Harness Engineering report documents roughly one million lines of agent-generated code with zero human-written lines, driven not by prompt wording but by durable control artifacts—`AGENTS.md` specifications, architectural rules, and linters repurposed as a compiler that rejects any output violating project invariants [OpenAI, 2026].

Harness Engineering nonetheless rests on a deeper, usually unstated assumption: *one Harness corresponds to one continuously evolving codebase*. The Harness automates the act of judging, but the code it judges is still treated as a long-lived artifact—it has a main branch, a release cadence, and accumulating technical debt; it is versioned, reviewed, and merged exactly as human-written code always has been. The Harness thus governs a single implementation that must be maintained indefinitely. This assumption is the one the present work removes: §3 unbinds the Harness from any single codebase, treating code as a regenerable snapshot rather than a persistent asset—and broadens

what a Harness carries, from the specification-plus-evaluation pairing assumed here to one that adds a knowledge prior and a curriculum of Milestones.

3 ForgeTrain: Method

ForgeTrain is our methodology for forging a production training framework end-to-end, and the training-framework instance of *Forge Engineering*—the code an agent forges is a disposable artifact, while the apparatus that produces it is the lasting asset.

3.1 Overview: The Agent Loop and the Golden Reference

ForgeTrain forges an entire production training framework with an **agent loop** that runs with no human inside it. An autonomous coding agent generates, runs, and refines candidate implementations and optimizations on its own, iteration after iteration—no human writes a line of the framework, and no human steers the loop prompt by prompt or vibe-codes it along; once started, it closes on itself.

What such a closed loop needs is a signal, available at every step, that tells it whether a candidate is right, and ForgeTrain's insight is to obtain that signal for free: *do not define correctness from scratch—appropriate an implementation already known to be correct, and hold it as ground truth*. We take a trusted, correct framework—such as Megatron-LM [Shoeybi et al., 2019], PyTorch [Paszke et al., 2019], or DeepSpeed [Rajbhandari et al., 2020]—as a *golden reference*: an artifact we hold as ground truth—it *defines* what is correct, and serves at once as *oracle* and *yardstick*—and it is what keeps the loop honest, fixing what *correct* means and adjudicating every candidate the loop produces. The framework emerges as the reference's *refinement*, along a chain of equivalence relations relaxed monotonically from bit-level identity down to training-quality parity. The agent loop and the golden reference that disciplines it are the foundation the rest of the method builds on.

Instantiating ForgeTrain for a new model and target framework asks little of the user: define the reference-implementation script that drives the golden reference, set the system parameters that describe the target through a single TOML configuration, and start the loop. From there it runs as two cooperating agents. A **develop agent** analyzes the task assigned to the current round and implements it autonomously against the Harness. A **review agent** sits over every change: it verifies that the develop agent has neither bypassed the evaluation nor edited the Harness itself, so progress is earned by passing the checks rather than by weakening them. The develop agent iterates round after round—implement, submit to review, revise—until it judges every task complete and the review agent concurrently confirms it.

With the reference and the loop in place, what remains is what the loop must cross. From an empty repository to a production-grade framework, the loop must answer three questions: how to keep the forged framework *correct*, how to push it to *peak performance*, and whether such a framework *can be optimized into existence by the loop at all*. ForgeTrain pairs each question with one mechanism: the three-stage spine together with Milestones holds correctness (§3.2), the Gate drives throughput toward the peak target through round-by-round measured adjudication (§3.3), and the knowledge prior keeps the climb from stalling halfway up (§3.4).

3.2 Correctness: The Three-Stage Spine and Milestones

A framework must reproduce the golden reference exactly to be trustworthy, yet diverge from it to be faster; since correctness is defined as agreement with the reference, every optimization perturbs the quantities that define correctness. This is not a constraint to be solved in place but a pair of mutually exclusive demands—no fixed target is at once "equal to" and "better than" the reference. The spine resolves the tension by separating the two demands *in time*: it descends a lattice of equivalence relations to the reference—from bit-level identity to training-quality parity—relaxing what the loop must satisfy at each step while never relaxing correctness itself (Table 2). This explicit progression—reproduce bit-for-bit, then relax the equivalence to surpass—has not, to our knowledge, been articulated as a methodology: prior reference-anchored work uses the reference only for differential testing, and reference-free synthesis has no anchor to tighten against.

Table 2: The spine descends an equivalence lattice to the golden reference, relaxing the binding constraint monotonically while preserving correctness.

Stage	Equivalence demanded	Freedom granted	Success criterion
A: Anchor	—	—	Captures the right invariants
B: Bit-for-Bit	Bit-level identity	None beyond the reference	Exact match at every anchor
C: Surpass	Training-quality parity	Restructure, fuse, rewrite	Quality parity, higher throughput

3.2.1 Stages: Anchor, Bit-for-Bit, Surpass

The first stage builds the oracle; the next two descend the lattice.

Stage A (Anchor) turns the golden reference into a machine-checkable oracle: running it on the target scenario, we capture the artifacts that determine training dynamics—activations, gradients, optimizer states, the loss-scaling trajectory, the collective-communication pattern—as assertions, together with the invariants over their ordering (a given all-reduce before a given parameter update, say). Anchors are taken at per-operator granularity, at exact numerical values, and over the edge cases production must survive but unit tests omit (gradient overflow and loss-scale backoff, checkpoint save-and-restore across a parallelism change, the first step after resumption); weaker anchors admit an implementation correct under normal inputs yet silently wrong in production.

Stage B (Bit-for-Bit) clears those anchors with exact equivalence: given identical inputs and seeds, every anchor matches bit for bit. We demand this over the usual loss-curve agreement because it rules out agreement for the wrong reasons—canceling errors, an unstable kernel passing on a benign input, a wrong reduction order hidden inside an acceptable loss. Optimization is forbidden here, which separates correctness from performance. Our current single-node, eight-GPU ForgeEngine build clears the full anchors at roughly 80% of Megatron-LM's throughput—slower, but provably computing the reference's result.

Stage C (Surpass) drops one notch to *training-quality parity*—the forged model need only match the reference's loss trajectory and downstream evaluation within statistical noise. Divergence is safe because Stage B established that the implementation computes the reference's intended quantities, so the develop agent may now restructure freely—fusing operators, reordering reductions, overlapping communication, rewriting recomputation policies. These moves break the bit-level anchors, so the enforcement of correctness passes from the anchors to the Gate of §3.3: every candidate optimization must survive a real training run compared against the baseline, admitted only when the measured deviation stays within a controlled bound. The result matches Megatron-LM on training quality while running 5–10% faster on equivalent hardware: the correctness and speed that could not be pursued at once, delivered in sequence. We defer the breakdown to §5.

3.2.2 Milestones: Recursive Decomposition of Each Climb

The three stages separate "equal to" from "better than" in time, but they shorten neither road: the loop must still climb from an empty repository to bit-for-bit identity, and again from a correct-but-slow baseline to a surpassing one, and neither climb is remotely a single attempt—the distance is too great, and mid-climb no signal tells the loop whether a partial attempt is on track. *Milestones* close this distance inside each stage of the spine. A Milestone is an intermediate, golden-reference-verified target, and the decomposition is recursive—a long climb breaks into legs, each leg into shorter legs, until every step is short enough for the agent to complete and the reference to check.

Milestones are therefore not a mechanism beside the spine but how the spine unfolds within its stages. In Stage B they plant operator-level checkpoints along the climb to bit-for-bit correctness; in Stage C they plant intermediate throughput targets along the climb to surpassing. Each Milestone is guarded by a Gate of §3.3—verified against the golden reference before the next leg begins—so the climb is not merely decomposed but checked leg by leg: correctness is not accepted once at the finish line but held continuously at every waypoint.

3.3 Performance: A Throughput Climb Driven by the Gate

The spine fixes what *kind* of equivalence each stage demands, and Milestones break the climb into reachable legs—but neither answers where performance comes from. In Stage C the Gate's bar is set at an MFU target far above the current implementation, and no single optimization can clear it in one move: the develop agent must sprint at it across many rounds—each round produces a candidate optimization and submits it to the *Gate* for adjudication, a real training run that compares training quality and throughput at once. Across this multi-round search, the Gate helps the agent in three ways. *Measured positioning*: every adjudication is a real run returning the candidate's exact readings on quality and throughput, so the agent knows each round how far it stands from the target—never leaning on an optimistic estimate. *Actionable failure*: a rejection does not merely say no; it returns a localized discrepancy—training quality out of bounds, or speedup short of the mark—that points directly at what to change next round. *Safe accumulation*: optimizations that pass are locked into the baseline and become the starting point of every later round, while failed changes are rolled back whole—progress ratchets forward only, so the agent can dare aggressive rewrites without fear of destroying what it has already won. Measured MFU thus climbs toward the target round by round (Figure 1c): the performance history of a forge is an ascending trajectory adjudicated into existence, every percentage point backed by a real run.

Gates are fine-grained: the loop carries one gate per task—and one per Milestone a task is broken into—so across a full forge they far outnumber the three stages. Each candidate is admitted only by passing the gate of the task it serves. The stage sets only what kind of equivalence a gate demands (bit-for-bit at the anchors in Stage B, training-quality parity against the baseline in Stage C); the mechanism is uniform across all of them.

An autonomous loop optimizing against a measured target has every incentive to game the measurement rather than meet it—all the more when the measurement is its own promotion criterion. Two design choices close that door.

Fully codified. Every requirement we place on the infrastructure—the bit-for-bit anchors, the training-quality bound, the throughput baseline, the conditions a run must satisfy—is expressed as executable check code rather than prose, so admission is a deterministic verdict the loop cannot argue with.

Permission-protected. The develop agent may edit only the framework under construction; it cannot touch the version-control history or the Harness itself, since rewriting either would clear the gate by hacking the evaluation instead of optimizing against it. The Harness's own permission management enforces this boundary, and the review agent backstops it by auditing each change for such evasions. The review agent alone may edit the gate—never to relax correctness, but to lower a target set too aggressively that would otherwise leave the develop agent spinning against an unreachable bar.

Every step of the climb is therefore earned: the Gate decides what is allowed to land. But the Gate cannot decide what the loop *dares to attempt*—an adjudicator that only admits safe changes cannot stop the search itself from settling somewhere safe. That is the final question.

3.4 Optimizability: The Knowledge Prior

The spine makes optimization *safe*, and the Gate measures every step's gain faithfully—but both answer whether a candidate should be admitted, not where worthwhile candidates come from. The remaining question is thus the most fundamental: can such a framework be *optimized into existence* by an agent loop at all? Once Stage C lifts bit-identity, the develop agent searches the space of legal rewrites as a learning process—probing, retreating from regressions, following improvement—and converges. The trouble is *where*: the rewrites that approach the peak are multi-step, with worse intermediate states. A fusion pays off only after the memory layout is also changed; a reduction reorder helps only once communication is overlapped. Facing a valley it must cross before the gain appears, the search relaxes into the nearest local optimum—and the Gate will faithfully admit every step of its staying there, yet never take the next step for it.

The failure is not a capability gap. A frontier coding agent can write each of these rewrites; what it will not do is commit to a sequence whose early steps look unrewarding—a shortfall of nerve rather than knowledge, which more code cannot fix. The *knowledge prior* addresses this by raising the search's effective *learning rate*: a durable corpus of optimization experience the agent consults—which aggressive directions pay off in a given regime, which intermediate costs are worth tolerating,

which changes compound together—that emboldens the agent to accept a stretch of worse steps and climb out of the local optimum an unaided search relaxes into. Every bolder attempt still passes the Gate, so the prior raises ambition without risking correctness: the prior governs what the loop dares to attempt, the Gate what is allowed to land. The prior itself never writes code; it charts the space, and because it lives in the Harness rather than the forged framework, it is part of the apparatus rather than the implementation—consistent with the zero-human-written-code discipline.

The three mechanisms are now each in place, and the questions they answer are orthogonal. Milestones shorten the *distance*: they add no optimization knowledge, only a route that keeps each step within reach. The Gate holds *each step*: it adjudicates whether a candidate both preserves the equivalence and advances throughput. The knowledge prior lifts the *ceiling*: it decides how high the search dares to aim. Together they make the golden-reference refinement executable by an imperfect agent—the prior sets how high to aim, Milestones make every step attainable, and the Gate rules on whether each step counts.

4 ForgeEngine: Forged Per-Scenario Training Frameworks

4.1 The Forged Artifact

ForgeTrain does not yield a single framework generalized across models; faithful to the Forge Engineering discipline, it forges a dedicated *ForgeEngine* for each target scenario, beginning from an empty repository. We report two such engines, forged independently for MiniCPM4 0.5B and MiniCPM4 8B against Megatron-LM [Shoeybi et al., 2019] as the golden reference. Each is a complete training stack rather than an isolated kernel: it implements the collective communication of the parallel groups, optimizer-state management, and the mixed-precision policy, and operates end to end from data ingestion through the optimizer step. The parallelism each engine carries is scoped to its target: the 0.5B engine runs under data parallelism alone, while the 8B engine adds tensor parallelism—each scenario receives exactly the parallel strategy it needs and no more.

The forged code is compact and the forging cheap. The 0.5B engine comprises roughly $35\mathrm{K}$ lines authored entirely by the agent loop—$12{,}235$ lines of orchestration at the framework layer plus $22{,}718$ lines of hand-forged operators (five GEMM variants and a custom FlashAttention forward kernel)—about one quarter of Megatron-LM's full codebase ($139{,}656$ lines under `megatron/`) and one third of its core ($106{,}030$ lines). Reaching a complete framework-level implementation took roughly *ten hours*; completing the engine in full, including operator refinement, took two to three days. The compactness holds by construction: the engine carries no historical branches, no abstraction layers amortized across other models, and no configuration surface for scenarios it will never encounter—every line exists solely for the one model-and-hardware pair it was forged against, so the line-count comparison understates, rather than inflates, the gap to a general-purpose framework.

4.2 A Forging Episode: From Anchor to Surpass

This section walks the three-stage spine in the order the forging itself runs. For the Anchor and Bit-for-Bit stages we make it concrete by following a single component—one transformer layer's forward pass together with the all-reduce that synchronizes its gradients; for the Surpass stage we widen to the whole engine and present what the agent actually discovered, along the framework line and the operator line.

Anchor. The golden reference is run on the target scenario and the layer's behavior is recorded: its forward activations, the gradients flowing into its parameters, and the communication pattern of the associated all-reduce. Each becomes a machine-checkable assertion, and their ordering—the all-reduce must complete before the corresponding parameter update—becomes an invariant the Harness enforces independently of how the layer is implemented.

Bit-for-Bit. The forged layer must reproduce these anchors exactly: under identical inputs and seeds, its activations and gradients match the reference to the bit. Any deviation is reported as a localized defect to be fixed, never an optimization to be kept; optimization is forbidden at this stage by construction. Once every component clears its anchors to this standard, the stage delivers its product: a framework in full alignment with Megatron-LM—slower, but provably computing the reference's result.

Surpass. Once bit-level identity has established that the layer computes the reference's intended quantities, the equivalence relaxes to training-quality parity and the agent is free to restructure the layer—fusing its operators, choosing a kernel per shape, overlapping its all-reduce with the computation that follows—and, beyond this one layer, the whole engine, each change admitted only after the gate confirms that training quality is preserved. What the agent does with that freedom is the subject of the rest of this section; what the freedom *is* deserves a word first, as it is easily mistaken.

We never prescribe which optimizations to apply. Instantiating ForgeTrain fixes only the golden reference and the equivalence that must be preserved; which transformations to attempt, in what combination, and how far to push each is left to the agent. This is essential rather than incidental: an engine told exactly what to do could at best reproduce the optimizations we already know, whereas only a search disciplined—not directed—by the gate can exceed the reference.

Yet this latitude is unprescribed, not unguided. As described in Sec. 3, the knowledge prior raises the effective learning rate of the search, pointing the agent toward aggressive, multi-step directions and emboldening it to commit to them. But a prior shapes a search without fixing its outcome—as an initialization or a learning rate shapes gradient descent without supplying the answer: it carries generic, portable optimization experience, not the scenario-specific optimizations themselves, nor the code that realizes them, nor how they compose and tune for this model and hardware. Those the agent discovers by searching against the gate. Prescription would fix the destination; the prior only quickens the journey.

Under that freedom, the Surpass stage unfolds along two lines: one at the *framework* level, applying optimizations the reference does not perform on top of the framework Bit-for-Bit delivered; and one at the *operator* level, taking the operators used during Bit-for-Bit as the baseline and forging replacements for them.

4.2.1 Framework-Level Optimizations

Operating across the whole engine rather than a single layer, the *Surpass* stage admitted, on top of the bit-for-bit framework, a family of framework-level optimizations, each cleared by the gate. Set against the golden reference, these optimizations fall into exactly three classes: those the reference has and the agent rebuilt along similar lines, those the reference has but for which the agent drew a different implementation boundary, and those the reference lacks and the agent discovered on its own. The floor comes from the first class; the win over the reference comes from the latter two.

The first class rebuilds the reference's optimization surface—the performance floor the engine needs to stand beside the reference: bucketed gradient reduce-scatter overlapped with the backward pass (bucket size tuned to the scenario over several ablation rounds), a ZeRO-1-style sharded optimizer with BF16 parameter all-gather, multi-tensor optimizer and gradient clipping, a family of fused kernels covering cross-entropy, SwiGLU, residual-RMSNorm, and RoPE (with precomputed cos/sin tables), and asynchronous checkpoint writing. Each has a counterpart in Megatron-LM, and the agent's implementations follow the same lines at comparable performance—this class explains why the engine does not lose to the reference, not why it wins.

The second class pursues goals the reference also pursues, but the agent drew different boundaries on measured returns:

- **Whole-forward graph capture.** The roughly three hundred kernel launches from the embedding to the output projection are captured into a single CUDA graph replayed once per step, with the backward pass, communication, and the optimizer left outside. Megatron-LM offers a wider graph spectrum—per-layer graphs, a full-iteration graph that includes gradient reduction, and a separately graphed optimizer step; the agent's narrower boundary is an adjudicated outcome: the wider candidates (forward-plus-backward graphs, collectives and optimizer in-graph) were each tried in earnest and rejected by the gate for negative measured returns or an unsupporting runtime stack.
- **Fused optimizer step.** Gradient clipping, the AdamW update, and the master-to-`bf16` write-back are fused into a single kernel (`fused_clip_adam_sync`), where Megatron-LM keeps the three as separate steps.
- **Optimizer–gather pipeline.** On the 8B engine, the parameter all-gather is interleaved with the optimizer step bucket by bucket: the moment a bucket's Adam update completes, the all-gather of

its BF16 parameters is issued on the communication stream, ordered by forward consumption so it overlaps the next step's forward, which waits only at bucket boundaries on the matching events. Megatron-LM's `overlap_param_gather` likewise overlaps the all-gather with the next forward, but dispatches it lazily, bucket by bucket, from forward pre-hooks; the agent moved the dispatch into the optimizer step itself, interleaved with per-bucket Adam as a single pipeline.

The third class has no Megatron-LM counterpart:

- **Asynchronous loss all-reduce.** The scalar loss all-reduce is issued on the communication stream and overlapped with the optimizer step, hiding it entirely in measurement; Megatron-LM leaves this reduction synchronously on the critical path.
- **Device-side gradient norm and clipping.** The gradient norm, clip coefficient, and scaling are computed entirely on device—l2norm, all-reduce, square root, clamp, and scale in one uninterrupted pipeline—where Megatron-LM reads the norm back to the host (`.item()`) between norm and clip, placing a CPU–GPU synchronization on the hot path.
- **Zero-copy gradient views.** Having established that its fused Adam kernel only reads gradients, the agent had the all-gather return views rather than copies, eliminating roughly one hundred and fifty clones and about 2 GB of peak memory per step.
- **Per-shape operator selection.** A dispatcher benchmarks candidate implementations across CuTeDSL, cuBLAS, Triton, and Transformer Engine and selects, per operator shape, the kernel of highest model-FLOPs utilization (MFU); Megatron-LM relies on a fixed backend with a small set of flags and performs no such automatic selection.
- **High-priority auxiliary streams.** The streams carrying wgrad and gradient reduction are set to high CUDA priority, letting them land ahead of the main stream's large GEMMs at SM scheduling boundaries and shortening the exposed communication and wgrad tail at the end of each step; Megatron-LM runs all its streams at default priority.

None of the three classes contains anything novel in isolation; what matters is that this distribution was adjudicated, not prescribed. The same search also left behind the rejected half—forced NCCL algorithm tuning, the wider graph captures, several layout rewrites that introduced trajectory drift—each faithfully caught and rolled back by the gate. The agent rebuilt the reference's floor, redrew boundaries on the reference's mechanisms, and found gains beyond the reference; the gate decided what stayed.

4.2.2 Kernel-Level Optimizations

The framework-level optimizations above schedule and fuse operators while treating each as a unit; a second, finer line of work forges the operators themselves, with the operators used during Bit-for-Bit as its baseline. The same *Surpass* freedom lets the agent rewrite the handful of operators that dominate the step's throughput budget at the target shapes (§5.4)—five GEMM variants (the projection and FFN matrix multiplies) and a custom FlashAttention forward and backward kernel—rather than treat any vendor library as fixed. Each forged kernel must clear the same bit-for-bit anchors as every other change before it is admitted, and is kept only if it improves on the per-shape selection of Sec. 4.2.1. The rewrites recur across operators and fall into four families:

- **Warp specialization and software pipelining.** Threads within a block are split into producer roles that move data and consumer roles that compute, with every pipeline stage multiply-buffered and the constituent matrix multiplies issued asynchronously, so memory movement and computation overlap across stages rather than serializing. A general-purpose library fixes one thread-to-work mapping for a whole shape class; forging it for the engine's exact dimensions recovers the occupancy that mapping leaves on the table.
- **Memory-layout specialization.** On-chip data is laid out for the operator's exact shape—flattening accumulator buffers to remove shared-memory bank conflicts, vectorizing the load/store paths, and sourcing matrix-multiply operands directly from registers rather than staging them through shared memory. These are the layout choices a vendor kernel cannot hard-code without knowing the shape, and they convert otherwise-stalled memory pipes into useful throughput.
- **Backward-pass fusion.** An operator's backward is computed by two matrix multiplies—one for the data gradient, one for the weight gradient—and, under sharding, a gradient reduction; the agent

folds all three into a single kernel, sparing the launch overhead and intermediate write-back that a vendor library's separate `dgrad`, `wgrad`, and reduce kernels pay.

- **Cross-kernel scheduling.** Adjacent kernels are overlapped across their launch boundary through programmatic dependent launch, and computation that the problem structure renders redundant is pruned outright (for a causal operator, evaluating the mask only on the diagonal block). Both recover time that lives between or inside kernels, where a single-kernel benchmark cannot reach.

As at the framework level, none of these is novel in isolation; what matters is that the agent discovered which ones the target shapes reward, composed them, and cleared each against the gate. The per-operator measurements, the head-to-head comparison against the vendor kernels, and a representative forging trajectory are deferred to §5.4.

5 Experiments

5.1 Setup

Hardware. All experiments run on NVIDIA H100 GPUs, in nodes of eight interconnected by NVLink. We evaluate the 0.5B ForgeEngine on 64 GPUs and the 8B ForgeEngine on 8 GPUs within a single node.

Software. Our stack is PyTorch 2.8.0, CUDA 12.9, cuDNN 9.10.02, and NCCL 2.27.3. The forged engines build on Transformer Engine 2.4, and their hand-forged operators target CUTLASS 4.5.0. We take Megatron-LM 0.15.0 [Shoeybi et al., 2019] as the golden reference throughout.

Models and configuration. We report two scenarios, the standard MiniCPM4 0.5B and 8B models (architectural details in Appendix B). Both use a sequence length of 4096 and Megatron-LM's mixed-precision training recipe; per-scenario parallelism and batching are given in Table 3. The 8B configuration has not been exercised in a production training run.

Table 3: Per-scenario evaluation configuration.

Scenario	GPUs	Parallelism	Micro-batch	Global batch	Seq. len
MiniCPM4 0.5B	64	Data-parallel only	10	1280	4096
MiniCPM4 8B	8	TP=2	4	128	4096

Metric. We report throughput as model-FLOPs utilization (MFU), computed from an exact per-operator FLOP count rather than the usual closed-form approximation (the full expression is given in Appendix C). Downstream evaluation benchmarks are introduced where they are used (§5.5).

5.2 Bit-for-Bit Correctness

Stage B makes the strongest correctness claim in this paper: the forged engine does not merely train a comparable model, it reproduces the golden reference's computation to the bit. We make that claim machine-checkable rather than aspirational.

What is anchored, and how. We do not hand-pick the tensors to compare. The agent instruments the golden reference with forward and backward hooks, dumps the tensors that pass through them under a shared input and seed, and replays the same input through the forged engine, comparing every captured tensor bit for bit at zero tolerance. The unit of comparison is therefore the individual tensor—every activation and gradient the hooks observe—rather than a coarse end-of-step summary. Because the anchor set is *discovered* by the agent rather than prescribed by us, its exact size depends on what the agent chooses to hook; Table 4 reports the two scenarios.

Table 4: Bit-for-bit anchor counts, as captured by the agent's own hooks on the golden reference. Every listed tensor must match the reference exactly (zero tolerance).

Scenario	Forward	Backward	Total
MiniCPM4 0.5B	183	339	522
MiniCPM4 8B	164	195	359

For the 8B engine the anchors decompose cleanly: a few global tensors (the embedding among them) plus a fixed per-layer set replicated across all 32 transformer layers—$4+5\times32=164$ on the forward pass and $3+6\times32=195$ on the backward—alongside roughly ten backward diagnostic intermediates that aid fault localization but are excluded from the acceptance count. The 0.5B engine carries *more* anchors despite being the smaller model: its backward hooks dump both activation gradients and weight gradients, so the totals track the agent's hooking choices rather than model size.

Why bit-for-bit. We demand bit-level identity over the usual loss-curve agreement because it rules out agreement for the wrong reasons—canceling errors, an unstable kernel that happens to pass on a benign input, a wrong reduction order hidden inside an acceptable loss. A single deviating bit at any anchor fails the stage and is surfaced as a localized defect to repair, never an optimization to keep: optimization is forbidden here by construction, which is what separates correctness from performance.

Result. Both engines clear their full anchor sets at zero tolerance, so the forged computation is provably the reference's computation before any performance work begins. The price of that guarantee is speed: with optimization still forbidden, the bit-exact 0.5B engine sustains 30.45% MFU—about 75.6% of Megatron-LM's 40.3% on the same hardware, the "roughly 80%" cited in §3.2.1. This correct-but-slow engine is exactly the baseline from which the throughput climb of §5.3 departs.

5.3 Framework-Level Throughput

Stage B leaves the agent with an engine that is correct but slow: it reproduces the reference's quantities bit-for-bit (§5.2) yet sustains only 30.45% MFU on the 0.5B scenario, far short of the 40.3% that Megatron-LM reaches on the same hardware. Stage C's task is to close and then overturn that gap without disturbing the numerics Stage B locked in. The agent loop does so incrementally: at each step it profiles the live engine, proposes a single throughput change, and admits it only if the Stage-C gate certifies that training quality is preserved (§5.5). Figure 2 plots the resulting trajectory—each marker is one accepted optimization, and the full step-by-step list is tabulated in Appendix A—and two qualitatively distinct regimes emerge.

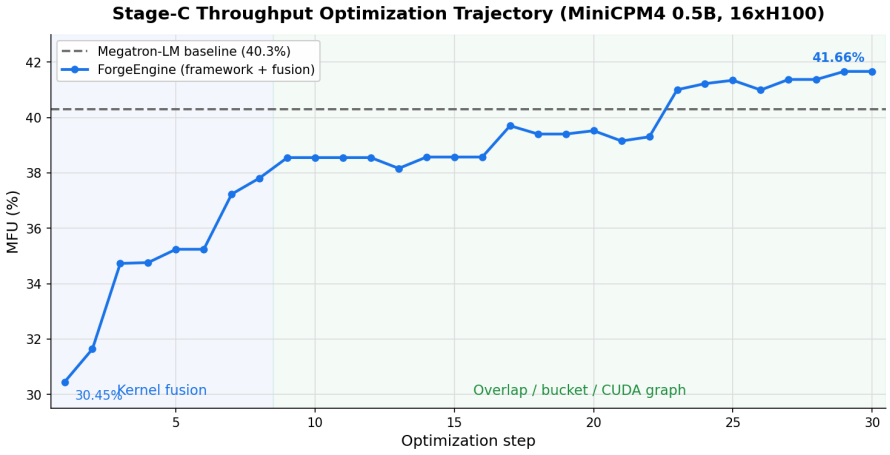

Figure 2: Stage-C throughput-optimization trajectory for MiniCPM4 0.5B (16×H100). Starting from the Stage-B baseline (30.45% MFU), the agent loop climbs through a kernel-fusion phase and an overlap/bucket/CUDA-graph phase to a framework-level peak of 41.66%, crossing the Megatron-LM reference (40.3%). Kernel-level forging (§5.4) adds further gains on top of this trajectory.

Kernel fusion. The first regime is compute-bound: the unoptimized engine dispatches one kernel per tensor operation, so launch overhead and redundant memory traffic dominate. The agent attacks this by fusing elementwise, normalization, and optimizer work into compound kernels—a fused cross-entropy and SwiGLU, fused residual-add and RMSNorm, a fused gradient-clip/Adam/parameter-sync step, and an `as_strided` cross-entropy fold into the final projection—while repairing the numerical edge cases (a cross-entropy `NaN`, an embedding-backward path) that naive fusion exposes. These eight steps lift MFU from 30.45% to 37.81%, recovering most of the gap to the reference purely by removing per-operator overhead.

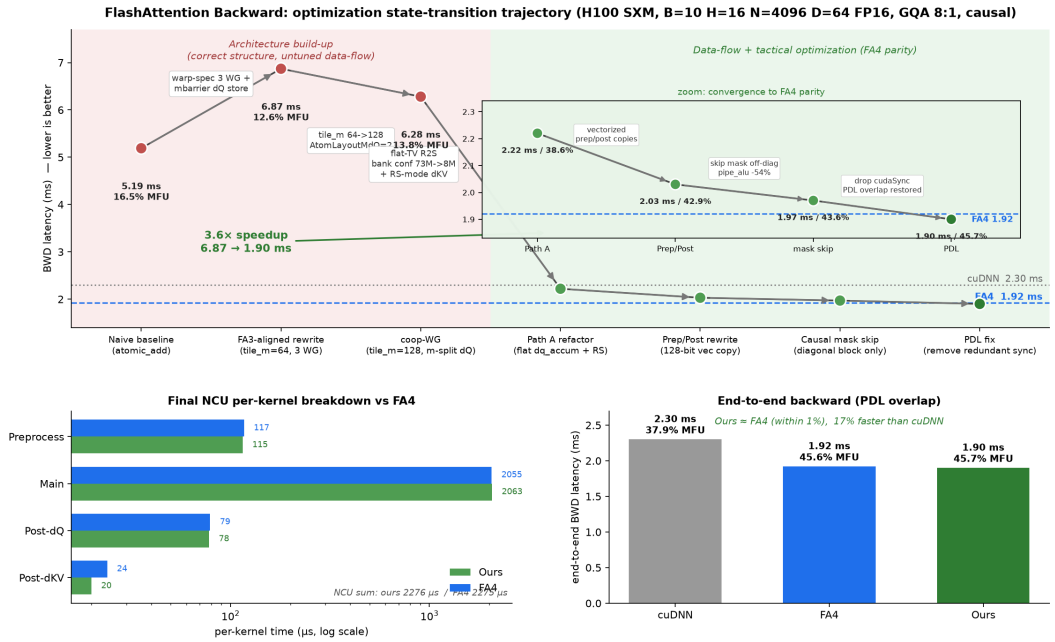

Figure 3: Optimization trajectory of the FlashAttention backward kernel (B=10, H=16, N=4096, D=64, FP16, GQA 8:1, causal; H100 SXM). **Top:** backward latency (lower is better) across milestones. A structural build-up phase (warp-specialized three-kernel pipeline, 128-row M-tile, double-buffered stages) is mutually dependent and regresses latency to 6.3–6.9 ms before a dQ data-layout refactor (flat global buffer, register-to-shared copy, register-sourced GEMM operands) drops the kernel to 2.22 ms; three independent tactical steps—128-bit vectorized preprocess/postprocess copies, diagonal-only causal masking, and removing a stray synchronization that had disabled programmatic dependent launch—then reach 1.90 ms (45.7% MFU). **Bottom left:** per-kernel NCU time breakdown versus FlashAttention-4 (forged sum 2276 µs vs. 2275). **Bottom right:** end-to-end backward latency—the forged kernel matches FA-4 to within 1% and is 17% faster than cuDNN.

Communication overlap and graph capture. Once the kernels are dense, the engine becomes communication- and launch-bound, and the second regime targets those costs. The agent introduces gradient bucketing with overlapped all-gather and optimizer updates, tunes NCCL and makes the loss all-reduce asynchronous, shards optimizer state (ZeRO-1), fuses the RoPE and normalization backward passes, and finally captures the full training step into a CUDA graph with progressively larger gradient buckets. This regime carries MFU from 37.81% past the Megatron-LM reference to a framework-level peak of 41.66%. The forged engine thus surpasses the hand-tuned baseline using only framework-level changes; the kernel-level forging of §5.4 compounds on top of this result.

5.4 Kernel-Level Forging: GEMM and FlashAttention

The framework-level climb of §5.3 treats each operator as a unit to schedule and fuse; a second, deeper line of forging rewrites the operators themselves. This is where most of the engine's hand-forged code lives—the 22,718 kernel lines of §5.6—and where the forged kernels are pitted directly against the strongest vendor libraries on the engine's own shapes. We forge five GEMM variants (the projection and FFN matrix multiplies) and a custom FlashAttention forward and backward kernel, the operators that dominate the training step at the target shapes.

Operator budget. Which operators are worth forging is an empirical question, so we measure it. Table 5 profiles a steady-state training step of the golden reference (Megatron-LM, backed by cuBLAS and Transformer Engine) on a single H100 at the 0.5B shape, decomposing GPU time by operator class. Two buckets dominate the compute: the GEMMs at 55.9% and the attention core at 16.6%, together 72.5% of the step. The elementwise bucket—normalization, RoPE, activations, and the Adam update—accounts for a further 24.4%, but it is memory-bound rather than compute-bound and is addressed by the fusion and graph-capture work of §5.3 rather than by kernel forging. This is a

Table 5: Operator time budget of a steady-state training step (Megatron-LM golden reference; single H100, MiniCPM4 0.5B, micro-batch 10, sequence 4096, bfloat16; forward+backward+optimizer, averaged over 5 profiled steps). Share is of GPU compute time, communication excluded.

Operator class	**Share of step**
GEMM (`qkv`/`o`/`fc1`/`fc2`/`lm-head`, fwd+dgrad+wgrad)	55.9%
Attention core (cuDNN FlashAttention fwd+bwd)	16.6%
Elementwise (RMSNorm, RoPE, activation, Adam)	24.4%
Other (Triton concat, sort)	3.1%

Table 6: Forged-kernel throughput (MFU, %) at the 0.5B engine's own shapes (H100, bfloat16; weight gradients accumulated in FP32). **(a)** attention forward and backward across vendor implementations; **(b)** the five forged GEMMs as forged/cuBLAS pairs. The projection (`qkv`) and output backward passes are single fused dgrad–wgrad–reduce kernels; "n/a" marks component passes the fused kernel does not expose, against which the three-kernel cuBLAS sequence is still measured separately. Bold marks the best value in each comparison—per row in (a), per forged/cuBLAS pair in (b)—with ties within 0.1 pp both bold.

(a) Attention

Direction	**Forged**	**cuDNN**	**FA-3**	**FA-4**	**TE**
Forward	42.6	**46.6**	46.3	41.4	39.0
Backward	**44.8**	35.3	**44.9**	42.6	36.2

(b) GEMM (forged / cuBLAS)

Operator	**Forward**	**Backward**	**dgrad**	**wgrad**
`qkv`	69.7/**69.9**	68.4/**73.7**	**72.9**/**72.8**	62.4/**71.6**
`attn-out`	**70.8**/69.8	**73.0**/71.5	**69.9**/**69.9**	**72.7**/69.3
`fc1`	**77.1**/74.2	**81.4**/**81.5**	81.9/**82.1**	**83.3**/81.1
`fc2`	80.2/**80.5**	**79.9**/78.9	**76.9**/**76.9**	**83.3**/81.7
`output`	67.5/**67.9**	**73.6**/70.4	n/a / 75.9	n/a / 66.1

single-GPU profile with no communication; at scale the data-parallel collectives are overlapped with this compute (§5.3) and do not enlarge the budget. The budget therefore points kernel-level forging squarely at the GEMMs and the attention core, which is where we spend it.

Per-shape selection. No single backend is fastest across all the shapes a training step issues. Rather than fix one library, the engine carries an operator dispatcher that benchmarks the candidate backends—CuTeDSL, cuBLAS, Triton, and Transformer Engine—per shape and keeps the fastest. The engine therefore runs the per-shape lower envelope of all backends together with its own forged kernels, rather than committing to any one backend's curve.

Correctness. The forged kernels are not exempt from Stage B: each must clear the same bit-for-bit anchors as §5.2 before it is admitted, so outperforming a vendor kernel never trades correctness for speed—the kernel is first proven to compute the reference's result, then tuned.

Throughput against vendor kernels. Table 6 pits each forged kernel against the strongest vendor baseline at the engine's own shapes, and two patterns hold. On the GEMMs (Table 6b), the forged kernels track cuBLAS to within roughly a point everywhere and overtake it outright on several passes—the attention-output and FFN backward among them—while the two FFN matrices reach 80–83% MFU, close to the roofline for these shapes. The projection and output kernels fold their dgrad, wgrad, and the gradient reduction into a single pass, sparing the launch overhead and intermediate-gradient memory traffic that a three-kernel vendor sequence pays; the one shape where the forged kernel still trails is the `qkv` weight gradient (62.4% vs. 71.6%), the engine's remaining soft spot. On attention (Table 6a), the forged FlashAttention forward reaches 42.6% MFU, a few points behind cuDNN and FA-3 but ahead of FA-4 and Transformer Engine; the backward—which carries twice the forward's FLOPs (Appendix C) and is the kernel whose optimization trajectory Figure 3 traces—reaches 44.8% MFU, level with the best vendor backward (FA-3, 44.9%) and 9–10 points above cuDNN and Transformer Engine. The forging effort thus pays off exactly where the operator budget concentrates: the attention backward.

Anatomy of a forged kernel. The attention backward is also the clearest illustration of how the loop forges a kernel, and its trajectory (Figure 3) separates two kinds of decision. We optimize it on an H100 SXM at the engine's attention shape (B=10, H=16, N=4096, D=64, FP16, GQA 8:1, causal), organized—following the FA-3/FA-4 design—as a three-kernel pipeline (preprocess → main → postprocess). From a naive baseline that accumulates $\mathrm{d}Q$ through `atomicAdd` (5.19 ms, 16.5% MFU), the loop first lays a *structural* foundation: a 128-row M-tile (versus 64), warp specialization into one producer and two consumer warp-groups (384 threads, a 24/240-register split), double-buffering on every pipeline stage, and an asynchronous five-GEMM issue schedule. These choices are mutually dependent and do not pay off in isolation—while they are being put in place latency *regresses* to 6.3–6.9 ms, because the $\mathrm{d}Q$-accumulator data-flow is still the bottleneck. The decisive step is a data-layout refactor that replaces the WGMMA-fragment-aware $\mathrm{d}Q$ accumulator with a flat global buffer and a flat thread-value register-to-shared copy, and sources the $\mathrm{d}K/\mathrm{d}V$ GEMM operands directly from registers; this cuts shared-memory bank conflicts from 73 M to 8.2 M and brings the kernel to 2.22 ms (38.6% MFU). Three independent *tactical* optimizations then close the gap: 128-bit vectorized preprocess/postprocess copies (2.03 ms), skipping the causal mask on all but the diagonal block—which halves arithmetic-pipe instructions (−54%)—(1.97 ms), and removing a stray `cudaStreamSynchronize` that had disabled programmatic dependent launch, restoring cross-kernel overlap (1.90 ms, 45.7% MFU). The result matches FlashAttention-4 (1.92 ms, 45.6% MFU) to within 1% and is 17% faster than cuDNN (2.30 ms); the per-kernel NCU times sum to the same total as FA-4 (2276 versus 2275 µs), leaving cross-kernel launch overlap as the only end-to-end difference. The methodological point generalizes beyond this one kernel: structural decisions must be fixed jointly and up front, even through a transient regression, whereas tactical optimizations compose independently on top of a correct structure.

From kernels to the engine. The per-kernel numbers translate into an end-to-end gain through the operator budget of Table 5. Within a bucket, time scales as $\mathrm{FLOP}/\mathrm{MFU}$, so each forged kernel's share of its bucket shrinks by the ratio of its MFU to the vendor kernel the golden reference would run. On the attention core, replacing cuDNN (forward 46.6%, backward 35.3%) with the forged kernels (42.6% and 44.8%) cuts the bucket's time by 12.8%, weighting forward and backward by their 1:2 FLOP ratio (Appendix C); against the 16.6% attention budget this returns 2.1 pp of step time. On the GEMMs, the forged kernels run level with cuBLAS—the FLOP-weighted bucket speedup is only 1.0%, since the fused dgrad–wgrad–reduce pass saves launch and memory traffic the FLOP model does not see—returning a further 0.6 pp against the 55.9% GEMM budget. Together kernel-level forging removes 2.7% of step time, a 1.028× throughput gain that lifts the engine from the 41.66% framework-level peak of §5.3 to 42.8% MFU. Compounded with the framework-level climb over the Megatron-LM reference (40.3%), the framework- and kernel-level gains together reach 6.2%, with the attention core—where the budget is smaller but the forged backward decisively beats the vendor kernel—contributing the larger share.

5.5 Training-Quality Parity and End-to-End Validation

The throughput gains of §5.3–§5.4 are worth nothing if the forged engine trains a worse model. We close the loop on the 0.5B scenario: we pretrain MiniCPM4 0.5B from scratch with the forged ForgeEngine and verify that the resulting model matches one produced by the standard framework, first along the pretraining loss trajectory and then on downstream evaluation.

Training-quality parity. Under identical data and schedule, the ForgeEngine and Megatron-LM pretraining runs track the same loss trajectory within run-to-run variation; Figure 4 overlays the two over the decay phase, where the curves are indistinguishable to the end. This is precisely the Stage C gate criterion: once Stage B has established that the engine computes the reference's intended quantities, the optimizations of §5.3 and §5.4 are admitted only if they leave training quality intact, and the overlapping loss curves confirm they do.

End-to-end downstream parity. The released MiniCPM4 0.5B is itself pretrained with Megatron-LM, so the comparison is apples-to-apples: a 0.5B model pretrained with Megatron-LM against one pretrained with the forged ForgeEngine. We take both through the same supervised fine-tuning recipe and evaluate them on six standard benchmarks spanning Chinese knowledge (CMMLU, C-Eval), code (MBPP), mathematical reasoning (GSM8K, MATH), and instruction following (IFEval). As Table 7 shows, the two are at parity: the ForgeTrain-trained model matches or slightly exceeds the Megatron-trained baseline on five of the six benchmarks, for an average of 53.84 against 53.16

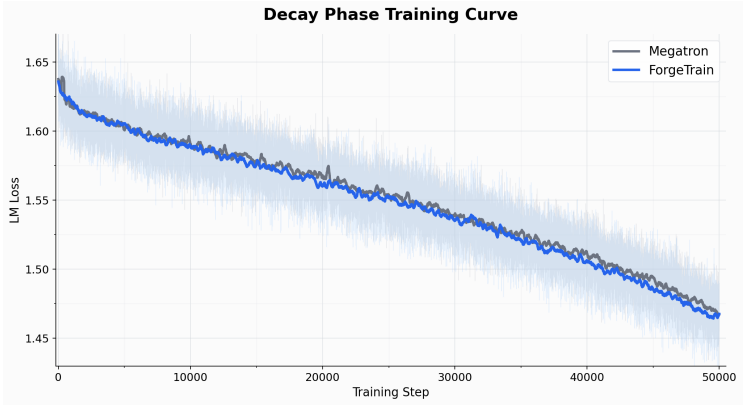

Figure 4: Decay-phase pretraining loss for MiniCPM4 0.5B: the forged ForgeEngine against the Megatron-LM reference. The two trajectories overlap within run-to-run variation throughout.

(+0.68), well within run-to-run variation. The entire training stack—collective communication, optimizer-state management, and the mixed-precision policy—was authored, integrated, and operated by the agent loop, yet sacrifices no model quality.

Table 7: Downstream evaluation after identical supervised fine-tuning: MiniCPM4 0.5B pretrained with Megatron-LM versus with the forged ForgeEngine.

Benchmark	**Megatron**	**ForgeTrain**	Δ
CMMLU	65.23	65.69	+0.46
MBPP (sanitized)	57.98	57.98	0.00
GSM8K	49.96	50.95	+0.99
MATH (PRM800K-500)	31.00	32.20	+1.20
IFEval	50.65	50.28	−0.37
C-Eval (CoT)	64.16	65.94	+1.78
Average	**53.16**	**53.84**	+0.68

5.6 Forging Cost

The premise of Forge Engineering—that forging from scratch is now cheap enough to prefer over reuse-and-extend—holds only if the cost is in fact low. We report it along two axes: wall-clock forging time and code volume.

Wall-clock. The 0.5B ForgeEngine reached framework-level completeness—a bit-for-bit-correct engine carried through the throughput climb of §5.3—in roughly ten hours of autonomous forging. Including the kernel-level work of §5.4, the full engine took two to three days. This stands against the multi-year effort of the dedicated team behind Megatron-LM; even granting that ForgeTrain forges a dedicated engine rather than a general framework, the difference is several orders of magnitude.

Code volume. The forged 0.5B engine is roughly 35K lines: 12,235 for the framework and orchestration layer and 22,718 for the hand-forged kernels (the five GEMM variants and the custom FlashAttention forward)—a complete training stack rather than a thin wrapper over a vendor library. Table 8 sets this against Megatron-LM: it is about a quarter of Megatron's `megatron/` tree and a third of its core. The comparison in fact understates the difference, since the forged engine carries no generality—no abstractions, configuration switches, or special-case branches for scenarios it does not serve—so every line is spent on the single model-and-hardware pair it targets.

Table 8: Code volume of the forged 0.5B ForgeEngine against Megatron-LM.

Codebase	**Lines**
ForgeEngine 0.5B — framework / orchestration	12,235
ForgeEngine 0.5B — hand-forged kernels	22,718
ForgeEngine 0.5B — total	**34,953**
Megatron-LM core	106,030
Megatron-LM (`megatron/`)	139,656

6 Related Work

6.1 Large-Scale Training Frameworks

ForgeTrain forges against a mature body of human-engineered training systems. Megatron-LM [Shoeybi et al., 2019] popularized tensor and pipeline model parallelism for transformers; DeepSpeed's ZeRO partitions optimizer states, gradients, and parameters across data-parallel ranks to fit trillion-parameter models [Rajbhandari et al., 2020]; GPipe introduced micro-batch pipeline parallelism with synchronous updates [Huang et al., 2019]; and Alpa automatically searches a hierarchical space of inter- and intra-operator parallel plans [Zheng et al., 2022]. These are the canonical incumbents of the one-codebase paradigm: a single general framework whose abstraction layers and configuration knobs are built to cover every model, scale, and hardware topology. Even Alpa's automation searches over a fixed framework's plan space rather than regenerating the framework itself. ForgeTrain inverts this: instead of one framework amortized across all scenarios, it forges a per-scenario ForgeEngine that refines against Megatron-LM as a golden model—matching it and ultimately running faster.

6.2 AI-Generated System Software

The closest neighbors to ForgeTrain are whole-system software artifacts written end-to-end by AI. VibeTensor [Xu et al., 2026] is presented as the first fully AI-generated deep learning system, a PyTorch-style eager runtime assembled by agents. It is, however, released for agentic-systems research only: it runs 1.7–$6.2\times$ slower than PyTorch, and its authors describe a "Frankenstein composition effect" in which locally correct components compose into a globally suboptimal whole. The shortfall is not implementation skill but the absence of a reference to anchor against—no truth source forces global behavior to match a production baseline.

Claude's C Compiler [Anthropic, 2026] is methodologically the closest relative: roughly one hundred thousand lines of Rust written by sixteen cooperating agents, compiling the Linux 6.9 kernel, with GCC used as a differential-testing oracle. Promoting a reference implementation to a golden reference is precisely the discipline ForgeTrain formalizes in its Anchor stage. Yet the compiler falls back to GCC for assembly and linking, is explicitly disclaimed as not production-ready, and targets the compiler domain rather than training infrastructure. ForgeTrain differs on all three axes these systems leave open: it is whole-system, production-grade, and matches *and* surpasses its reference on the reference's own benchmarks.

6.3 AI-Generated Kernels and Operators

A productive line of work forges performance-critical code at node granularity. AlphaEvolve [Google DeepMind, 2025] couples an evolutionary search loop with automated evaluation and has discovered scheduling heuristics and matrix-multiplication kernels deployed in production; KernelEvolve [Meta AI, 2026] evolves production-grade kernels under throughput targets; and Sakana's CUDA Engineer [Sakana AI, 2025] synthesizes thousands of test-verified CUDA kernels. The shared recipe—iterative or evolutionary search, test-driven verification, and profiling feedback—is precisely the Harness-driven loop, and these systems show it is already production-ready at the level of a single kernel or operator.

The distinction from ForgeTrain is granularity. Each of these efforts forges one self-contained node with a clear local objective, where correctness and speed can be measured in isolation. A training framework is an integrated system whose behavior—collective communication, optimizer-state

management, mixed-precision policy, and scheduling—is correct only globally, across an entire optimization trajectory. Forging at that granularity is what the three-stage refinement of §3 makes possible, and it is the gap these node-level efforts leave open.

7 Conclusion

ForgeTrain shows that production-grade system software—long the province of large, multi-year engineering efforts—can be forged end-to-end by an autonomous agent, provided the forging is disciplined by a trusted golden reference rather than a hand-written specification. The decisive move is to separate two demands that no fixed target reconciles: bit-level fidelity to the reference and the freedom to surpass it. Resolving them *in time*, along the Anchor–Bit-for-Bit–Surpass lattice with a gate that re-establishes correctness step by step once optimization resumes, turns "equal to" and "better than" from a contradiction into a sequence. A knowledge prior and recursively verified Milestones then carry the agent loop across the multi-step optimizations and the wide empty-repository-to-target gap it would not cross on its own. The forged ForgeEngines for MiniCPM4 0.5B and 8B reproduce Megatron-LM bit for bit and surpass it by 5–10% at matched training quality, and a from-scratch pretraining run confirms downstream parity—evidence that the entire training stack, from collective communication to mixed-precision policy, can be authored, integrated, and operated by AI alone.

The broader significance is positional. The training framework is the last major piece of the AI-building toolchain still authored by humans; forging it is precisely what crossing from L3 to L4 requires on the infrastructure axis of progress toward AGI, and ForgeTrain is a first concrete step across that boundary. It also instantiates a more general claim: once the marginal cost of writing code approaches zero, maintaining a single general codebase is no longer the rational default. Code becomes a disposable, per-scenario artifact, and the forging apparatus—the staged discipline, the knowledge prior, the Milestones—becomes the durable asset. We call this inversion *Forge Engineering* and expect it to recur wherever a trusted reference can anchor the forging.

Several directions follow. The same Harness should forge across hardware targets, retargeting an engine to a new accelerator without rewriting the reference. The approach should extend from training to inference frameworks, where scenario-specific optimization is equally costly. Most consequentially, closing the loop—using a ForgeEngine to train the next generation of the agent that forges it—would turn a one-time demonstration into a step on the recursive self-improvement frontier, the prospect that makes *authoring* infrastructure, rather than merely operating it, the decisive capability.

References

Dario Amodei. Machines of loving grace. Essay, 2024.

Anthropic. Demystifying evals for AI agents. Anthropic Engineering Blog, 2025.

Anthropic. Claude's C compiler. Anthropic Engineering Blog, 2026.

Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Yao Liu, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic discovery of optimization algorithms. *arXiv preprint arXiv:2302.06675*, 2023.

Ben Cottier, Robi Rahman, Loredana Fattorini, Nestor Maslej, and David Owen. The rising costs of training frontier AI models. *arXiv preprint arXiv:2405.21015*, 2024.

DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

Google DeepMind. AlphaEvolve: A coding agent for scientific and algorithmic discovery. Google DeepMind Blog, 2025.

John L. Hennessy and David A. Patterson. A new golden age for computer architecture. *Communications of the ACM*, 62(2):76–85, 2019.

Anson Ho, Tamay Besiroglu, Ege Erdil, David Owen, Robi Rahman, Zifan Carl Guo, David Atkinson, Neil Thompson, and Jaime Sevilla. Algorithmic progress in language models. *arXiv preprint arXiv:2403.05812*, 2024.

Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. GPipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Yixiu Liu, Yang Nan, Weixian Xu, Xiangkun Hu, Lyumanshan Ye, Zhen Qin, and Pengfei Liu. AlphaGo moment for model architecture discovery. *arXiv preprint arXiv:2507.18074*, 2025.

Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.

Meta AI. KernelEvolve: Evolving production-grade kernels via LLM. *arXiv preprint*, 2026.

OpenAI. A five-level framework for tracking progress toward AGI. Internal classification reported by Bloomberg, 2024.

OpenAI. Harness engineering. OpenAI Blog, 2026.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory optimizations toward training trillion parameter models. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2020.

Esteban Real, Chen Liang, David R. So, and Quoc V. Le. AutoML-zero: Evolving machine learning algorithms from scratch. In *International Conference on Machine Learning (ICML)*, 2020.

Sakana AI. The AI CUDA engineer. Sakana AI Technical Report, 2025.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. In *arXiv preprint arXiv:1909.08053*, 2019.

TechInsights. The $700 billion AI arms race: Hyperscaler capital expenditure in 2026. TechInsights Industry Analysis, 2026.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Hjalmar Wijk, Tao Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Josh Clymer, Jai Dhyani, et al. RE-bench: Evaluating frontier AI R&D capabilities of language model agents against human experts. *arXiv preprint arXiv:2411.15114*, 2024.

Bing Xu, Terry Chen, Tianqi Chen, Yangqing Jia, Vinod Grover, Wen-mei Hwu, Luis Ceze, and Humphrey Shi. VibeTensor: Our first fully AI generated deep learning system. *arXiv preprint arXiv:2601.16238*, 2026.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P. Xing, Joseph E. Gonzalez, and Ion Stoica. Alpa: Automating inter- and intra-operator parallelism for distributed deep learning. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.

Barret Zoph and Quoc V. Le. Neural architecture search with reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2017.

A Framework-Level Optimization Trajectory

Table 9 lists, in order of acceptance, the framework-level optimizations the agent loop applied to the MiniCPM4 0.5B ForgeEngine during Stage C, together with the measured MFU after each step (16×H100, DP only). This is the per-step detail behind the trajectory of Figure 2: the first eight steps form the kernel-fusion regime, and the remainder the overlap/bucket/CUDA-graph regime. Two candidates (†) were measured but rejected by the Stage-C gate and reverted, so they do not carry into the final engine.

Table 9: Framework-level optimizations applied to MiniCPM4 0.5B during Stage C, in acceptance order. MFU is measured on 16×H100 (data-parallel). † marks candidates that were reverted after the gate rejected them.

#	Phase	Optimization	MFU (%)
1	Kernel fusion	Stage-B baseline (no fusion)	30.45
2	Kernel fusion	Fused cross-entropy + SwiGLU	31.64
3	Kernel fusion	Cross-entropy `NaN` + embedding-backward fix	34.73
4	Kernel fusion	Direct Transformer-Engine attention	34.76
5	Kernel fusion	Fused residual-add + RMSNorm	35.24
6	Kernel fusion	Fused gradient-clip + Adam + param-sync	35.24
7	Kernel fusion	`as_strided` cross-entropy + FFN fusion	37.23
8	Kernel fusion	Cross-entropy prefill + fused RMSNorm backward	37.81
9	Overlap/graph	Gradient bucketing (100 MB)	38.55
10	Overlap/graph	Per-bucket all-gather + optimizer overlap	38.55
11	Overlap/graph	All-gather–optimizer overlap	38.55
12	Overlap/graph	Wgrad-overlap by default	38.55
13	Overlap/graph	NCCL tuning + async loss all-reduce	38.16
14	Overlap/graph	Production-default verification	38.57
15	Overlap/graph	Batch-flush write-grad restructure	38.57
16	Overlap/graph	Batch-flush gate	38.57
17	Overlap/graph	ZeRO-1 sharded optimizer	39.70
18	Overlap/graph	Sharded optimizer by default	39.40
19	Overlap/graph	Routing / baseline fixes	39.40
20	Overlap/graph	Fused norm-backward + multi-tensor Adam	39.52
21	Overlap/graph	Fused-operator micro-optimizations	39.15
22	Overlap/graph	RoPE strided-K (drop K copy)	39.30
23	Overlap/graph	Document-aware fused RoPE	41.00
24	Overlap/graph	Wgrad bucket 200 MB	41.22
25	Overlap/graph	RoPE backward in-place†	41.34
26	Overlap/graph	Forward-only CUDA Graph	40.99
27	Overlap/graph	Full-step CUDA Graph (phase A)	41.37
28	Overlap/graph	Full-step CUDA Graph (phase B/C)	41.37
29	Overlap/graph	Wgrad bucket 400 MB (single bucket)	41.66
30	Overlap/graph	Cross-step pipelining†	41.66

B Model Architectures

The two evaluation scenarios use the released MiniCPM4 0.5B and 8B models[1] without architectural modification. Both are decoder-only transformers with grouped-query attention, SwiGLU feed-forward blocks, pre-normalization RMSNorm, LongRoPE positional encoding, and the MiniCPM μP-style scaling (`scale_emb`, `dim_model_base`, `scale_depth`). Table 10 lists the configuration that fixes the FLOP count and the shapes the forged operators target; the experiments train at a sequence length of 4096 (§5.1), well within the 32768 context the architecture supports.

Table 10: MiniCPM4 architecture configuration for the two evaluation scenarios.

Hyperparameter	**MiniCPM4 0.5B**	**MiniCPM4 8B**
Transformer layers	24	32
Hidden size	1024	4096
FFN intermediate size	4096	16384
Attention heads	16	32
Key/value heads (GQA)	2	2
Head dimension	64	128
Vocabulary size	73448	73448
Activation	SwiGLU (SiLU)	SwiGLU (SiLU)
Normalization	RMSNorm (ϵ=10^{-5})	RMSNorm (ϵ=10^{-6})
Position encoding	LongRoPE	LongRoPE
Max context length	32768	32768
Tied input/output embeddings	Yes	No
μP (`scale_emb`, `dim_model_base`, `scale_depth`)	$12, 256, 1.4$	$12, 256, 1.4$
Training precision	bfloat16	bfloat16

C Model-FLOPs Utilization

We report MFU against an exact per-operator FLOP count rather than the usual closed-form $6ND$ approximation, which omits attention and miscounts the GQA projections and the vocabulary head. We count a multiply–accumulate as two FLOPs and report per token; a quantity is the same for every token in the batch, so the per-step numerator is this total times the number of tokens processed. Let h be the hidden size, f the FFN intermediate size, n_h and n_{kv} the query and key/value head counts, d the head dimension (h_q=$n_h d$, h_{kv}=$n_{kv} d$), V the vocabulary size, L the layer count, and s the sequence length. The forward per-token FLOPs of each operator class are

$$\begin{aligned}
\text{Attn. projections (Q,K,V,O)}: &\ 2h\,(h_q + 2h_{kv}) + 2h_q h,\\
\text{FFN (gate, up, down; SwiGLU)}: &\ 2\,(2hf) + 2fh,\\
\text{Attention core } (QK^\top,\ AV;\ \text{causal}): &\ 2n_h sd,\\
\text{Output / cross-entropy head}: &\ 2hV.
\end{aligned}$$

The attention core carries a factor $\frac{1}{2}$ for the causal mask (folded into the expression above), and grows with s where the others do not. The backward pass costs twice the forward for every GEMM (one matrix for the data gradient, one for the weight gradient) and twice the forward for the attention core, so each per-layer term is multiplied by three (one forward, two backward) and summed over L layers; the output head is counted once, also at the 1:2 forward-to-backward ratio. Table 11 evaluates this for the two evaluation scenarios.

Two properties of this count matter for the operator budget of §5.4. First, the GEMMs and the attention core together are the entire compute FLOP budget—normalization, RoPE, activations, and the optimizer contribute negligible FLOPs and are memory-bound, which is why they are absent from this table yet still consume measurable time (the elementwise bucket of Table 5). Second, the attention core is the one term that scales with s: at the 4096 sequence length it is 18.8% of the 0.5B FLOP budget, and two-thirds of that is backward, which is why the attention backward is the single operator whose forging trajectory we trace in detail (§5.4).

[1] `https://www.modelscope.cn/models/OpenBMB/MiniCPM4-0.5B` and `https://www.modelscope.cn/models/OpenBMB/MiniCPM4-8B`.

Table 11: Per-token FLOP budget (forward+backward) by operator class for the two MiniCPM4 scenarios at sequence length 4096, with each class's share of the per-step total. The total is the numerator of the MFU computed in §5.1.

	0.5B		8B	
Operator class	**GFLOP/tok**	**Share**	**GFLOP/tok**	**Share**
FFN GEMM (gate/up/down)	1.812	56.5%	38.655	76.5%
Attention core (FlashAttention)	0.604	18.8%	3.221	6.4%
Output / CE GEMM	0.451	14.1%	1.805	3.6%
Attn. projection GEMM (Q/K/V/O)	0.340	10.6%	6.845	13.5%
Total	3.207	100%	50.526	100%